

Assuming you mean SGLang, the simplest distinction is:

SGLang tells you what work it has given the GPU. Hardware tells you what the GPU is actually experiencing while doing that work.

Think of SGLang as a kitchen manager and the GPU as the kitchen:

- The manager knows how many orders are waiting, what each order contains, and how many meals are being prepared.
- Sensors inside the kitchen know whether the ovens are actually busy, whether workers are waiting for ingredients, whether equipment is overheating, and whether one workstation is slowing everything down.

What SGLang knows

SGLang understands the requests:

- How many requests are waiting.
- How many requests are running.
- How many prompt tokens must be processed.
- Whether a request is in prefill or decode.
- How much of the prompt is already cached.
- How much KV-cache space requests are consuming.
- How long requests have waited.
- Their priority, if provided.
- Their TTFT and output-token latency.

This is logical workload information.

For example, SGLang may know:

“I have 30 running requests, 50 waiting requests, and 80% of my KV-cache token slots are occupied.”

That is very useful, but it does not completely describe the physical GPU.

What hardware can tell you

1. Whether GPU compute units are actually busy

SGLang can estimate how much computation a batch should require.

Hardware counters tell you:

- How many GPU multiprocessors are active.
- How heavily Tensor Cores are being used.
- Whether the GPU is executing efficiently.
- Whether GPU cores are spending time waiting rather than calculating.

SGLang might say:

“I submitted a large batch, so the GPU should be busy.”

Hardware might reveal:

“Only 35% of the GPU’s compute capacity is being exercised.”

That could happen because the batch shape is inefficient, kernels are too small, the CPU is not feeding the GPU quickly enough, or one GPU is waiting for another.

2. Whether memory bandwidth is the bottleneck

This is different from simply asking, “How much GPU memory is allocated?”

The GPU may have enough memory capacity but still be spending most of its time moving data from HBM.

Hardware can tell you:

- How active GPU memory reads and writes are.
- Whether memory bandwidth is approaching saturation.
- Whether compute units are waiting for data.
- Hardware-cache behavior, such as L2 hits and misses.

This distinction matters because LLM phases behave differently:

- Prefill is frequently more compute-intensive.
- Decode is frequently more dependent on memory bandwidth.

SGLang knows which phase it is running, but hardware tells you whether that particular workload is actually compute-bound or memory-bound on your GPU.

3. Actual physical memory state

SGLang knows its own memory structures:

- Model memory.
- Logical KV-cache token usage.
- Cached prefixes.
- Available KV pages.

Hardware can provide:

- Total HBM capacity.
- HBM allocated and free.
- Memory-bandwidth activity.
- Memory errors.
- Memory usage from other processes.

There is an important nuance: SGLang may reserve a large block of GPU memory in advance. Therefore, the hardware may report that 90% of HBM is allocated even though SGLang still has many free KV-cache slots.

So:

- Use SGLang to determine whether another request can fit in its KV pool.
- Use hardware telemetry to understand total physical HBM pressure and actual bandwidth pressure.

4. Throttling and physical problems

SGLang may notice that inference has become slower, but it may not know why.

Hardware can expose:

- GPU temperature.
- Power limits.
- Thermal throttling.
- Reduced clock speeds.
- ECC errors.
- PCIe congestion.
- NVLink congestion.
- A slow GPU within a tensor-parallel group.

For example, SGLang may observe that the same batch now takes 20% longer. Hardware could reveal that the GPU clock was reduced because it reached a power or temperature limit.

No amount of queue inspection alone would accurately reveal that.

Why this is especially important for agentic workflows

Agentic workflows do not continuously use the model. They frequently alternate between inference and external tools:

- 1 The agent calls the model.
- 2 The model requests a tool.
- 3 The agent waits for the tool for several seconds.
- 4 The tool result returns.
- 5 The agent calls the model again with a longer context.

During the tool call, that agent is not occupying the GPU.

Imagine 100 agents:

- 80 are waiting for tools.
- 10 are waiting in SGLang.
- 10 are currently running on the GPU.

A few seconds later, 50 tools might complete simultaneously. Suddenly, 50 new model requests arrive.

This creates a highly irregular pattern:

Model activity → tool wait → model activity → tool wait → burst of activity

SGLang can see requests once they arrive, but it usually cannot know:

- How many tools are still executing.
- When those tools will return.
- Whether 50 agents are about to resume.
- Which returning agent has the most urgent user-facing request.

The GPU hardware also cannot know those future events. It only knows its current physical state.

That means an effective agentic system needs three kinds of information:

Layer	Information it provides
Agent harness	Priority, tool-wait state, expected tool completion, session identity and SLO
SGLang	Queue, prompt tokens, prefix reuse, running batch and KV-cache state
GPU hardware	Actual compute activity, memory bandwidth, HBM state, clocks and throttling

A concrete example

Suppose SGLang reports:

- 40 waiting requests.
- 20 running requests.
- KV-cache usage at 60%.

That does not tell you whether more requests should be admitted.

Hardware telemetry could show one of two situations.

Situation A: GPU has physical headroom

- SM activity: 35%.
- Tensor Core activity: 25%.
- Memory-bandwidth activity: 30%.
- No throttling.

Interpretation:

The GPU is not being fully exercised. SGLang may be able to admit more requests or increase its prefill batch.

Situation B: GPU is memory-bandwidth saturated

- SM activity: 80%.
- Tensor Core activity: 30%.
- Memory-bandwidth activity: 90%.
- Output-token latency is increasing.

Interpretation:

The GPU is already struggling to move model and KV-cache data. Adding more decode requests could worsen token latency, even though KV capacity is only 60% occupied.

SGLang sees the same queue and KV usage in both situations. Hardware reveals that the correct scheduling decision is different.

The important limitation

Hardware telemetry should not replace SGLang's scheduler information.

Hardware tells you:

“The GPU is currently memory-bandwidth saturated.”

But hardware does not tell you:

- Which request is urgent.
- Which request has a cached prefix.
- Which request has waited longest.
- Which request needs only 100 prompt tokens.
- Which request belongs to an agent whose tool just completed.
- Which request is approaching its TTFT deadline.

Therefore, the ideal design is:

Use SGLang to understand and order the requests. Use hardware information to determine how aggressively SGLang should admit and batch them.

In one sentence:

SGLang understands the work; hardware measures the physical consequences of that work; the agent harness understands why the work matters.